

Paradygmaty Programowania

Laboratorium 3

Przykładowe użycie wzorców projektowych

1 Wstęp teoretyczny

Przed przeczytaniem niniejszego wstępu zapoznaj się z materiałami do wykładu 3, 4 oraz 5.

1.1 Cel ćwiczenia

W trakcie ćwiczenia studenci będą projektować diagramy klas UML dla prostego zadania, korzystając z odpowiednich wzorców projektowych. Celem jest praktyczne zastosowanie wiedzy na temat diagramów klas i wzorców projektowych, a także rozwijanie umiejętności analizy problemów.

1.2 UML i wzorce projektowe – przypomnienie

UML (Unified Modeling Language) to standardowy język modelowania i projektowania systemów informatycznych. Ułatwia komunikację pomiędzy programistami, analitykami, architektami oraz innymi uczestnikami projektu, umożliwiając graficzną reprezentację struktury i zachowań systemu. Diagramy klas UML to jeden z najczęściej stosowanych rodzajów diagramów, służących do przedstawienia struktury systemu za pomocą klas, ich atrybutów, metod oraz relacji między nimi.

Wzorce projektowe to sprawdzone, wielokrotnie używane rozwiązania dla określonych problemów projektowania oprogramowania. Wprowadzenie wzorców projektowych do projektu może usprawnić rozwój, ułatwić zrozumienie i modyfikację kodu, a także poprawić jego jakość.

Diagramy klas UML służą do przedstawienia statycznej struktury systemu. Reprezentują klasy, interfejsy, ich atrybuty, metody oraz relacje między nimi. Na diagramach klas można wyróżnić:

- Klasy: symbolizowane jako prostokąty z trzema sekcjami (nazwa klasy, atrybuty, metody)
- Interfejsy: oznaczane przez podwójną linię podkreślającą nazwę interfejsu, lub jako prostokąt z etykietą «interface»

- Atrybuty: składowe klasy opisujące jej cechy
- Metody: funkcje, które realizują określone zadania w klasie
- Relacje: linie łączące elementy, reprezentujące różne rodzaje związków, takie jak dziedziczenie, asocjacja, agregacja czy kompozycja

Wzorce projektowe można podzielić na trzy główne kategorie:

- Wzorce kreacyjne: opisują proces tworzenia obiektów, np. Singleton, Fabryka, Prototyp
- Wzorce strukturalne: dotyczą kompozycji klas i obiektów, np. Adapter, Most, Fasada, Dekorator
- Wzorce behawioralne: definiują współpracę między obiektami i ich komunikację, np. Łańcuch odpowiedzialności, Iterator, Obserwator, Strategia

1.3 Opis wybranych wzorców projektowych

Poniżej zostały krótko opisane wybrane¹ wzorce projektowe. Pierwsze sześć odnosi się do wzorców omawianych na wykładzie. Rozwiązanie zadania powinno być możliwe z wykorzystaniem tych wzorców. W celu poszerzenia horyzontu w podobny sposób przedstawione zostały też inne wzorce, często występujące w systemach informatycznych.

1.3.1 Strategia (Strategy)

Problem: Masz do czynienia z różnymi wariantami algorytmu lub zachowania w ramach jednej rodziny klas. Chcesz umożliwić zmianę algorytmów w trakcie działania programu, bez wprowadzania zmian w klasach klienta i utrzymania elastyczności oraz rozszerzalności kodu.

Rozwiązanie: Wzorzec Strategia polega na definiowaniu interfejsu, który opisuje wspólne zachowanie dla wszystkich strategii w danej rodziny algorytmów. Następnie tworzysz klasy konkretne, które implementują ten interfejs, reprezentując różne warianty algorytmów. Klasa klienta zawiera referencję do obiektu interfejsu strategii i komunikuje się z nim poprzez ten interfejs. W trakcie działania programu klient może zmieniać strategię poprzez podmianę referencji na inny obiekt konkretnej klasy strategii. Dzięki zastosowaniu wzorca Strategia, algorytmy są łatwo zamienne, a klient nie musi być modyfikowany przy wprowadzaniu nowych wariantów algorytmów.

1.3.2 Kompozyt (Composite)

Problem: Reprezentowanie hierarchicznej struktury obiektów, gdzie obiekty mogą zawierać inne obiekty tego samego rodzaju.

Rozwiązanie: Wzorzec Kompozyt pozwala na traktowanie pojedynczych obiektów i ich złożzeń w sposób jednolity. Wprowadza on wspólny interfejs dla obiektów pojedynczych i złożonych. W przypadku obiektów złożonych, metody tego interfejsu są implementowane w taki sposób, by uwzględniać ich składowe.

¹Nie opisano wszystkich znanych wzorców. Niektóre z nich wychodzą z użycia, inne powstają.

1.3.3 Dekorator (Decorator)

Problem: Potrzeba rozszerzenia funkcjonalności obiektów bez wprowadzania zmian do istniejących klas.

Rozwiązanie: Wzorzec Dekorator wprowadza abstrakcyjną klasę dekoratora, która implementuje ten sam interfejs co dekorowany obiekt. Dekoratory przekazują wywołania metod do dekorowanego obiektu, a następnie wykonują dodatkowe akcje. Pozwala to na dynamiczne dodawanie funkcjonalności do obiektów.

1.3.4 Fabryka Abstrakcyjna (Abstract Factory)

Problem: Masz do czynienia z systemem, który wymaga tworzenia rodziny powiązanych lub zależnych obiektów, ale nie chcesz zobowiązywać klienta do korzystania z konkretnych klas tych obiektów. Wprowadzenie nowych rodzajów produktów może wpłynąć na zmianę klienta i zmniejszyć elastyczność oraz rozszerzalność systemu.

Rozwiązanie: Wzorzec Fabryka Abstrakcyjna polega na definiowaniu interfejsu fabryki, który deklaruje metody do tworzenia powiązanych lub zależnych obiektów, ale nie określa ich konkretnych klas. Następnie tworzysz konkretne klasy fabryk, które implementują ten interfejs i mają odpowiedzialność za tworzenie konkretnej rodziny produktów. Klient korzysta z interfejsu fabryki do tworzenia obiektów i nie musi znać szczegółów dotyczących implementacji konkretnych klas produktów.

Dzięki zastosowaniu wzorca Fabryka Abstrakcyjna, klient oddziela proces tworzenia obiektów od ich konkretnych klas, co pozwala na łatwe wprowadzenie nowych rodzajów produktów i rodziny produktów. Wzorzec ten zwiększa elastyczność i rozszerzalność systemu, ponieważ klient nie musi być modyfikowany przy wprowadzaniu nowych wariantów produktów.

1.3.5 Budowniczy (Builder)

Problem: Konstrukcja obiektów składających się z wielu komponentów, z opcjonalnymi lub złożonymi elementami, może prowadzić do zawiłych konstruktorów.

Rozwiązanie: Wzorzec Budowniczy wprowadza oddzielny obiekt, który tworzy obiekty złożone krok po kroku. Budowniczy definiuje interfejs określający metody do konstrukcji obiektów. Klient korzysta z dyrektora, który zarządza procesem budowy, używając konkretnego budowniczego do tworzenia obiektów.

1.3.6 Polecenie (Command)

Problem: Konieczność oddzielenia obiektów wywołujących operację od obiektów, które te operacje wykonują. Pragniemy również umożliwić zapisywanie, kolejkovanie lub cofanie operacji.

Rozwiązanie: Wzorzec Polecenie wprowadza abstrakcyjną klasę lub interfejs reprezentujący operację. Konkretnie klasy poleceń implementują interfejs i zawierają logikę do wykonania operacji. Obiekty wywołujące operacje korzystają z interfejsu poleceń, co pozwala na

zmianę wykonywanej operacji w trakcie działania programu.

1.3.7 Metody wytwórcza (Factory Method)

Problem: Chcemy stworzyć obiekty różnych klas, które mają wspólny interfejs, lecz nie chcemy bezpośrednio używać operatora "new". Fabryka pozwala na utworzenie obiektów różnych klas bez związania ich z konkretnymi klasami, zwiększając elastyczność kodu.

Rozwiązanie: Wzorzec Fabryka opiera się na definiowaniu interfejsu fabryki, który zawiera metodę do tworzenia obiektów. Konkretnie fabryki implementują ten interfejs i decydują, jakie obiekty mają być tworzone. Klient korzysta z interfejsu fabryki, zamiast bezpośrednio z konkretnej klasy.

1.3.8 Łańcuch odpowiedzialności (Chain of Responsibility)

Problem: Chcemy uniknąć sprzężenia bezpośredniego między obiektami, które wysyłają żądania i obiektami, które je obsługują. Żądanie powinno być przekazywane między różnymi obiektami, dopóki nie zostanie obsłużone przez odpowiedni.

Rozwiązanie: Wzorzec Łańcuch odpowiedzialności definiuje interfejs obsługujący żądania, który zawiera odniesienie do następnego obiektu w łańcuchu. Każdy obiekt w łańcuchu sprawdza, czy może obsłużyć żądanie; jeśli nie, przekazuje je do następnego obiektu. Pozwala to na dynamiczne przekazywanie żądań między obiektami.

1.3.9 Obserwator (Observer)

Problem: Wiele obiektów musi być informowanych o zmianach stanu innego obiektu, bez utrzymywania silnych powiązań między nimi.

Rozwiązanie: Wzorzec Obserwator definiuje jeden-do-wielu zależność między obiektami. Gdy stan obiektu ulega zmianie, wszystkie zależne obiekty są powiadamiane. Wprowadza abstrakcyjne klasy lub interfejsy dla obserwatorów oraz obiektów obserwowanych. Obserwatorzy rejestrują się w obserwowanym obiekcie, który powiadamia ich o zmianach.

1.3.10 Stan (State)

Problem: Chcemy zmieniać zachowanie obiektu w zależności od jego wewnętrznego stanu, bez modyfikacji klasy tego obiektu.

Rozwiązanie: Wzorzec Stan wprowadza interfejs reprezentujący różne stany obiektu. Konkretnie klasy stanów implementują ten interfejs i zawierają logikę dla swojego stanu. Obiekt przechowuje referencję do swojego aktualnego stanu i deleguje mu odpowiednie operacje. Zmiana stanu obiektu polega na zmianie referencji do obiektu stanu.

1.3.11 Most (Bridge)

Problem: Chcemy oddzielić abstrakcję od jej implementacji, aby można było je modyfikować niezależnie. W ten sposób unikamy stałego powiązania między hierarchią klas abstrakcji

a hierarchią klas implementacji.

Rozwiązanie: Wzorzec Most wprowadza interfejs implementacji, który reprezentuje implementację abstrakcji. Abstrakcja zawiera odniesienie do interfejsu implementacji i przekazuje do niego wywołania metod. W efekcie abstrakcja i implementacja są niezależne i można je modyfikować bez wpływu na siebie.

1.3.12 Adapter (Adapter)

Problem: Chcemy umożliwić współpracę między obiektami o niekompatybilnych interfejsach, bez modyfikacji ich kodu.

Rozwiązanie: Wzorzec Adapter wprowadza klasę adaptera, która implementuje interfejs oczekiwany przez klienta, a jednocześnie ma dostęp do obiektu o niekompatybilnym interfejsie. Adapter tłumaczy wywołania metod klienta na wywołania zgodne z obiektem adaptowanym.

1.3.13 Fasada (Facade)

Problem: Chcemy uprościć interakcje klientów z podsystemem składającym się z wielu klas o skomplikowanych zależnościach i interfejsach.

Rozwiązanie: Wzorzec Fasada wprowadza jednolity interfejs, który ukrywa złożoność podsystemu przed klientem. Fasada udostępnia uproszczone metody, które są łatwe do zrozumienia i używania przez klientów, a wewnętrznie korzysta z klas podsystemu.

1.3.14 Singleton (Singleton)

Problem: Chcemy zagwarantować, że dana klasa posiada tylko jedną instancję, a jednocześnie dostęp do niej jest łatwy.

Rozwiązanie: Wzorzec Singleton ukrywa konstruktor klasy, uniemożliwiając tworzenie wielu instancji. Zamiast tego, udostępnia globalny punkt dostępu do swojego obiektu. W przypadku potrzeby tworzenia nowego obiektu, wzorzec Singleton zawsze zwraca istniejącą instancję.

1.3.15 Prototyp (Prototype)

Problem: Chcemy tworzyć kopie obiektów bez związania z konkretnymi klasami tych obiektów.

Rozwiązanie: Wzorzec Prototyp wprowadza interfejs do klonowania obiektów, który muszą implementować wszystkie klasy, które mają być klonowane. Klient korzysta z interfejsu prototypu, zamiast bezpośrednio z konkretnych klas, co pozwala na tworzenie kopii obiektów różnych klas.

1.3.16 Pylek (Flyweight)

Problem: Potrzeba zarządzania dużą liczbą obiektów, które mają wspólne elementy, co prowadzi do nieefektywnego wykorzystania pamięci.

Rozwiązanie: Wzorzec Lekka waga wprowadza współdzielone obiekty, które przechowują wspólne elementy dla wielu obiektów. Lekkie obiekty zawierają jedynie unikalne, niezależne atrybuty. Wzorzec ten pozwala na efektywniejsze zarządzanie pamięcią i szybsze operacje na obiektach.

1.3.17 Mediator (Mediator)

Problem: Duża liczba powiązań między obiektami, prowadząca do trudności w zarządzaniu i utrzymaniu kodu.

Rozwiązanie: Wzorzec Mediator wprowadza obiekt mediatora, który zarządza komunikacją między obiektami, zamiast bezpośrednich powiązań między nimi. Obiekty komunikują się ze sobą tylko za pośrednictwem mediatora, co zmniejsza złożoność zależności i ułatwia modyfikacje.

1.3.18 Memento (Memento)

Problem: Chcemy przechowywać i przywracać wewnętrzny stan obiektu, bez naruszania jego enkapsulacji.

Rozwiązanie: Wzorzec Memento wprowadza obiekt memento, który przechowuje stan obiektu. Obiekt tworzący memento może tworzyć i przywracać swoje stany za pomocą obiektów memento, nie ujawniając swojej wewnętrznej struktury. Pozwala to na zaimplementowanie funkcjonalności takich jak cofanie operacji czy punkty przywracania.

1.3.19 Wizytator (Visitor)

Problem: Chcemy wykonywać operacje na zbiorze obiektów o różnych klasach, bez modyfikacji ich kodu. Istnieje potrzeba dodawania nowych operacji, nie wpływając na strukturę obiektów.

Rozwiązanie: Wzorzec Wizytator wprowadza interfejs wizytatora, który definiuje metody dla każdego rodzaju odwiedzanego obiektu. Konkretnie wizytatory implementują ten interfejs, zawierając logikę dla każdej operacji. Odwiedzane obiekty posiadają metodę akceptującą wizytatora, który wykonuje odpowiednią operację.

1.3.20 Interpreter (Interpreter)

Problem: Chcemy interpretować język lub reprezentację problemu, który można opisać za pomocą gramatyki lub drzewa abstrakcyjnego składni (AST).

Rozwiązanie: Wzorzec Interpreter definiuje gramatykę języka i wprowadza interfejsy dla klas reprezentujących elementy gramatyki. Konkretnie klasy implementują te interfejsy i za-

wierają logikę interpretacji. Klient tworzy drzewo abstrakcyjnego składni na podstawie danych wejściowych, a następnie przekazuje je do interpretera, który oblicza wynik.

1.3.21 Iteracyjny (Iterator)

Problem: Chcemy umożliwić sekwencyjne przeglądanie elementów kolekcji, nie ujawniając jej wewnętrznej struktury.

Rozwiązanie: Wzorzec Iteracyjny wprowadza interfejs iteratora, który definiuje metody do przeglądania i zarządzania elementami kolekcji. Konkretnie iteratory implementują ten interfejs, dostosowując go do konkretnej struktury danych. Klient korzysta z iteratora, zamiast bezpośrednio z kolekcji, co pozwala na ukrycie wewnętrznej struktury i uproszczenie kodu.

1.3.22 Prywatna klasa (Private Class Data)

Problem: Chcemy zminimalizować ryzyko przypadkowej modyfikacji danych w klasie poprzez ograniczenie dostępu do jej atrybutów.

Rozwiązanie: Wzorzec Prywatna klasa danych wprowadza klasę przechowującą dane, które mają być chronione. Ta klasa jest zagnieżdżona w klasie, której dane ma chronić, i ma dostęp do prywatnych atrybutów tej klasy. W ten sposób chronione dane są trudniejsze do modyfikacji przez przypadkowe błędy lub nieuprawnione operacje.

W powyższych opisach przedstawiono różnorodne wzorce projektowe, które mogą pomóc w rozwiązywaniu konkretnych problemów w trakcie projektowania i implementacji systemów. Warto zapoznać się z nimi dokładniej i stosować odpowiednio do potrzeb, aby tworzyć elastyczne, łatwe do modyfikacji i utrzymania aplikacje.

2 Przebieg ćwiczenia

Termin oraz sposób dostarczania sprawozdań należy uzgodnić z prowadzącym(i) zajęcia. Sprawozdanie powinno zawierać: **imię i nazwisko, numer albumu, datę i formułę:** *Oświadczam, że niniejsza praca stanowiąca podstawę do uznania osiągnięcia efektów uczenia się z przedmiotu Paradygmaty Programowania została wykonana przez mnie samodzielnie.*

Ćwiczenie przebiega według następującego harmonogramu:

- przedstawienie problemu programistycznego
- wspomagane przez prowadzących rozwiązywanie zadania

W celu przygotowania się do zajęć wystarczy przejrzeć materiały do wykładów 3, 4 oraz 5, ze zwróceniem szczególnej uwagi na:

- sposób przedstawiania klas na diagramach UML (statyczny klas),
- relacje jakie mogą zachodzić pomiędzy poszczególnymi klasami i sposób ich reprezentacji na diagramach UML (statyczny klas),

- zasady projektowania SOLID oraz
- wzorcami projektowymi zaprezentowanymi na wykładzie.

3 Przykładowe zadanie

Fragmenty prostej gry cRPG

W danej grze bohater, którego poczynaniami steruje gracz, porusza się po planszy złożonej z kafli reprezentujących różne rodzaje terenu. W grze występuje kilka rodzajów kafli i w przeszłości mogą się pojawiać nowe.

Dla każdego z kafli projektant gry przewidział kilka standardowych czynności, które bohater może wykonać na danym kafle, takie jak idź w określonym kierunku (przesuń bohatera na sąsiadujący kafel) czy odpocznij. Dodatkowo, każdy rodzaj kafli może mieć swoje unikalne czynności. Na przykład w lesie można spotkać dzikie zwierzę, które może zaatakować bohatera.

3.1 Zadanie 1 - 2 pkt

Zaprojektuj strukturę klas zawierającą co najmniej następujące klasy:

- Bohater
- Teren
- Mapa

Dla powyższych klas zaproponuj sposób dodawania nowych rodzajów terenu.

3.2 Zadanie 2 - 2 pkt

Projektowana gra ma być dostępna w dwóch wersjach:

- tekstowej, obsługiwanej z poziomu konsoli oraz
- graficznej, wyświetlającej w sposób uproszczony mapę i aktualną pozycję bohatera.

Zaprojektuj mechanizm wczytywania mapy z pliku dla obu wersji aplikacji.

3.3 Zadanie 3 - 2 pkt

W projektowanej grze podjęto próbę dodania kilku poziomów trudności. Poszczególne poziomy trudności różnią się siłą oraz częstotliwością pojawiających się przeciwników. Opisz jakiego wzorca projektowego należałoby użyć oraz uzasadnij swój wybór. Opisz jakie zmiany należałoby poczynić w istniejącej strukturze klas i jakie miałyby to konsekwencje dla projektu. Spróbuj narysować wybrany fragment diagramu klas z uwzględnieniem różnych poziomów trudności.

Ostatnia aktualizacja: 28 kwietnia 2025